

Foundations of Weight initialization

[Weight initialization]

1.0 Content Block

In deep learning, weight initialization or parameter initialization describes the initial step in creating a neural network. A neural network contains trainable parameters that are modified during training: weight initialization is the pre-training step of assigning initial values to these parameters. The choice of weight initialization method affects the speed of convergence, the scale of neural activation within the network, the scale of gradient signals during backpropagation, and the quality of the final model. Proper initialization is necessary for avoiding issues such as vanishing and exploding gradients and activation function saturation. Note that even though this article is titled "weight initialization", both weights and biases are used in a neural network as trainable parameters, so this article describes how both of these are initialized. Similarly, trainable parameters in convolutional neural networks (CNNs) are called kernels and biases, and this article also describes these.

2.0 Content Block

Glorot initialization Glorot initialization (or Xavier initialization) was proposed by Xavier Glorot and Yoshua Bengio. It was designed as a compromise between two goals: to preserve activation variance during the forward pass and to preserve gradient variance during the backward pass. For uniform initialization, it samples each entry in $W^{(l)}$ independently and identically from $U(\pm \sqrt{6 / (n_{l+1} + n_{l-1})})$. In the context, n_{l-1} is also called the "fan-in", and n_{l+1} the "fan-out". When the fan-in and fan-out are equal, then Glorot initialization is the same as LeCun initialization.

3.0 Content Block

Further reading Goodfellow, Ian; Bengio, Yoshua; Courville, Aaron (2016). "8.4 Parameter Initialization Strategies". Deep learning. Adaptive computation and machine learning. Cambridge, Mass: The MIT press. ISBN 978-0-262-03561-3. Narkhede, Meenal V.; Bartakke, Prashant P.; Sutaone, Mukul S. (June 28, 2021). "A review on weight initialization strategies for neural networks". Artificial Intelligence Review. 55 (1). Springer Science and Business Media LLC: 291–322. doi:10.1007/s10462-021-10033-z. ISSN 0269-2821.

4.0 Content Block

Constant initialization We discuss the main methods of initialization in the context of a multilayer perceptron (MLP). Specific strategies for initializing other network architectures are discussed in later sections. For an MLP, there are only two kinds of trainable parameters, called weights and biases. Each layer l contains a weight matrix $W^{(l)} \in \mathbb{R}^{n_{l-1} \times n_l}$ and a bias vector $b^{(l)} \in \mathbb{R}^{n_l}$, where n_l is the number of neurons in that layer. A weight initialization method is an algorithm for setting the initial values for $W^{(l)}, b^{(l)}$ for each layer l . The simplest form is zero initialization: $W^{(l)} = 0, b^{(l)} = 0$. Zero initialization is usually used for initializing biases, but it is not used for initializing weights, as it leads to symmetry in the network, causing all neurons to learn the same features. In this page, we assume $b = 0$ unless otherwise stated. Recurrent neural networks typically use activation functions with bounded range, such as sigmoid and tanh, since unbounded activation may cause exploding values. (Le, Jaitly, Hinton, 2015) suggested initializing weights in the recurrent parts of the network to identity and zero bias, similar to the idea of residual connections and LSTM with no forget gate. In most cases, the biases are initialized to

zero, though some situations can use a nonzero initialization. For example, in multiplicative units, such as the forget gate of LSTM, the bias can be initialized to 1 to allow good gradient signal through the gate. For neurons with ReLU activation, one can initialize the bias to a small positive value like 0.1, so that the gradient is likely nonzero at initialization, avoiding the dying ReLU problem.

5.0 Content Block

History Random weight initialization was used since Frank Rosenblatt's perceptrons. An early work that described weight initialization specifically was (LeCun et al., 1998). Before the 2010s era of deep learning, it was common to initialize models by "generative pre-training" using an unsupervised learning algorithm that is not backpropagation, as it was difficult to directly train deep neural networks by backpropagation. For example, a deep belief network was trained by using contrastive divergence layer by layer, starting from the bottom. (Martens, 2010) proposed Hessian-free Optimization, a quasi-Newton method to directly train deep networks. The work generated considerable excitement that initializing networks without pre-training phase was possible. However, a 2013 paper demonstrated that with well-chosen hyperparameters, momentum gradient descent with weight initialization was sufficient for training neural networks, without needing either quasi-Newton method or generative pre-training, a combination that is still in use as of 2024. Since then, the impact of initialization on tuning the variance has become less important, with methods developed to automatically tune variance, like batch normalization tuning the variance of the forward pass, and momentum-based optimizers tuning the variance of the backward pass. There is a tension between using careful weight initialization to decrease the need for normalization, and using normalization to decrease the need for careful weight initialization, with each approach having its tradeoffs. For example, batch normalization causes training examples in the minibatch to become dependent, an undesirable trait, while weight initialization is architecture-dependent.

6.0 Content Block

Others Instead of initializing all weights with random values on the order of $O(1/\sqrt{n})$, sparse initialization initialized only a small subset of the weights with larger random values, and the other weights zero, so that the total variance is still on the order of $O(1)$. Random walk initialization was designed for MLP so that during backpropagation, the L2 norm of gradient at each layer performs an unbiased random walk as one moves from the last layer to the first. Looks linear initialization was designed to allow the neural network to behave like a deep linear network at initialization, since $W \text{ReLU}(x) - W \text{ReLU}(-x) = Wx$. It initializes a matrix W of shape $R^{n/2 \times m}$ by any method, such as orthogonal initialization, then let the $R^{n \times m}$ weight matrix to be the concatenation of $W, -W$.